Звіт

до лабораторної роботи 1

з навчальної дисципліни

«Актуальні проблеми "Data Mining"»

Тема: Реалізація та порівняльний аналіз алгоритмів класифікації (1-Rule, Naive Bayes, Decision Tree, kNN).

Виконав: студент групи ММШІ-1 Красіков Антон Ігорович

1. Опис та постановка задачі

Метою даної роботи є повна програмна реалізація базових алгоритмів машинного навчання, подальше їх застосування до обраного набору даних та порівняння результатів між собою та з готовими імплементаціями моделей з бібліотеки scikit-learn.

Основні завдання:

- Реалізувати класифікатори: 1-Rule, Naive Bayes, Decision Tree, kNN.
- Провести попередню обробку даних (аналіз, масштабування, кодування).
- Дослідити вплив гіперпараметрів на результати навчання.
- Зробити висновок щодо доцільності використання кожного з класифікаторів на основі отриманих метрик.
- Провести порівняльний аналіз власних реалізацій з еталонними моделями зі scikit-learn за метриками якості та часом виконання.

2. Теоретичні відомості

Для вирішення задачі класифікації було обрано наступні математичні моделі:

- **1-Rule (One Rule):** Базовий (baseline) алгоритм, що генерує одне правило на основі найбільш інформативного атрибута, який мінімізує загальну похибку на навчальній вибірці.
- **k-Nearest Neighbors (kNN):** Метричний алгоритм «лінивого навчання». Класифікація нового об'єкта відбувається на основі мажоритарного голосування k його найближчих сусідів у багатовимірному просторі ознак (була використана Евклідова відстань).
- **Gaussian Naive Bayes:** Імовірнісний класифікатор, що базується на теоремі Байєса з «наївним» припущенням про незалежність ознак. Оскільки набір даних містить неперервні величини, було використано припущення про нормальний розподіл ознак.
- **Decision Tree:** Алгоритм побудови логічних правил у вигляді дерева. Розбиття у вузлах відбувається за принципом жадібної максимізації приросту інформації (Information Gain), що базується на зменшенні ентропії Шеннона.

3. Опис набору даних

Для проведення експериментів було використано набір даних, що містить соціально-демографічні показники та фактори навчального процесу студентів.

- **Загальний обсяг**: 2392 записи, які було розділено на навчальну (1913 об'єктів) та тестову (479 об'єктів) вибірки зі збереженням пропорцій класів.
- **Цільова змінна**: GradeClass - категоріальна змінна, що відображає підсумкову оцінку/рейтинг студента (5 унікальних класів від 0 до 4). У даних присутній виражений дисбаланс класів (мажоритарним є клас 4).
- **Ознаки**: До процесу моделювання було залучено 12 незалежних змінних:
    - **Неперервні та кількісні:** Вік (Age), час на самостійне навчання (StudyTimeWeekly), кількість пропусків занять (Absences).
    - **Категоріальні та бінарні:** Стать (Gender), етнічна приналежність (Ethnicity), рівень освіти батьків (ParentalEducation), рівень батьківської підтримки (ParentalSupport), наявність репетитора (Tutoring), а також участь у позакласних активностях (Extracurricular, Sports, Music, Volunteering).
- **Особливості препроцесингу**: З фінальної вибірки було вилучено ідентифікатор студента (StudentID), а також середній бал (GPA), оскільки він має пряму детерміновану залежність із цільовою змінною GradeClass. Після перетворення категоріальних ознак методом One-Hot Encoding фінальна розмірність простору ознак склала 29 колонок.

4. Хід роботи (пайплайн та архітектура)

Під час розробки системи був зроблений акцент на її масштабованості та коректності роботи з даними. Роботу можна поділити на наступні етапи:

1. **Архітектура моделей:** Створено абстрактний базовий клас BaseClassifier із жорстким контрактом методів fit(X, y) та predict(X). Усі чотири алгоритми реалізовано як його нащадків з використанням векторизованих операцій бібліотеки numpy (наприклад, обчислення відстаней через np.linalg.norm без вкладених циклів).
2. **Попередня обробка даних:** Було видалено неінформативні атрибути (ідентифікатори, імена, константні значення), які могли б призвести до перенавчання та хибних результатів.
    - Використано ColumnTransformer для роздільної обробки ознак: StandardScaler для неперервних числових величин (необхідно для коректної роботи kNN) та OneHotEncoder для категоріальних ознак. Цільова змінна кодувалась через LabelEncoder.
    - Трансформація даних застосовувалась після розділення на навчальну та тестову вибірки, щоб запобігти витоку інформації.
3. **Оцінка результатів:** Окрім базової метрики Accuracy, було реалізовано власні функції для розрахунку матриці помилок (Confusion Matrix) для власних та бібліотечних реалізацій, Precision, Recall та F1-score. Це дозволило більше об'єктивно оцінити здатність моделей розпізнавати міноритарні класи.
4. **Управління конфігурацією:** Управління гіперпараметрами (такими як кількість сусідів ***k*** або максимальна глибина дерева ***max_depth***) винесено в окремий файл *config.yaml.*

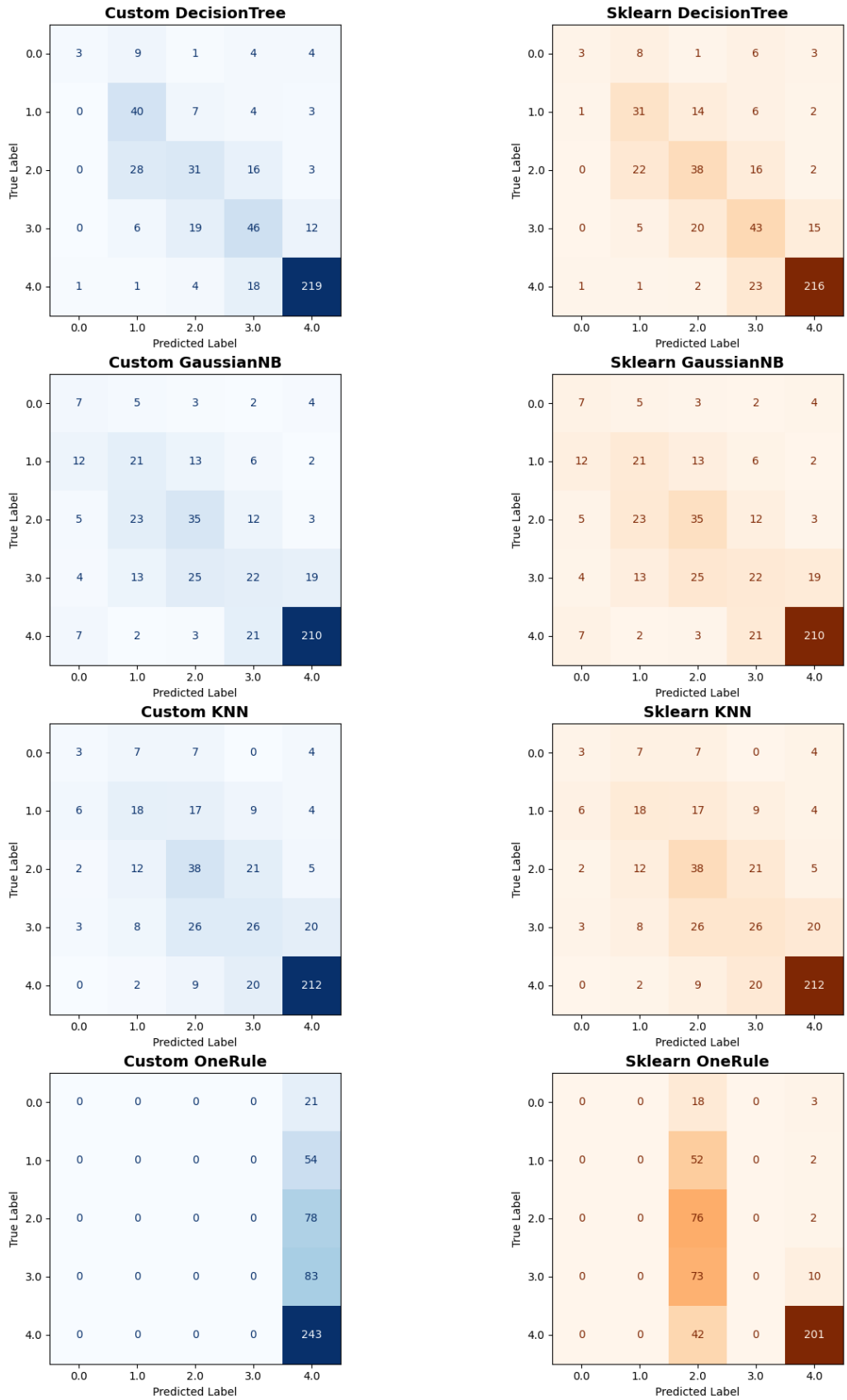
5. Оцінка результатів та висновки

Порівняння власних та бібліотечних моделей за різними показниками (accuracy, precision, recall, F1-score, fit time, pred time) на одному датасеті:

Model	Accuracy	Precision	Recall	F1-score	Fit Time (s)	Pred Time (s)
Custom DecisionTree	0.7077	0.6315	0.5473	0.5411	1.475989	0.000733
Sklearn DecisionTree	0.6910	0.5869	0.5222	0.5248	0.005315	0.000334
Custom GaussianNB	0.6159	0.4405	0.4600	0.4453	0.000583	0.043575
Sklearn GaussianNB	0.6159	0.4405	0.4600	0.4453	0.002032	0.000556
Custom KNN	0.6200	0.4393	0.4298	0.4316	0.000056	0.061795
Sklearn KNN	0.6200	0.4393	0.4298	0.4316	0.000574	0.002750
Custom OneRule	0.5073	0.1015	0.2000	0.1346	0.041637	0.000307
Sklearn OneRule	0.5783	0.2426	0.3603	0.2641	0.001850	0.000160

Для детального розуміння природи помилок кожного класифікатора, особливо в умовах дисбалансу класів, було побудовано матриці помилок:

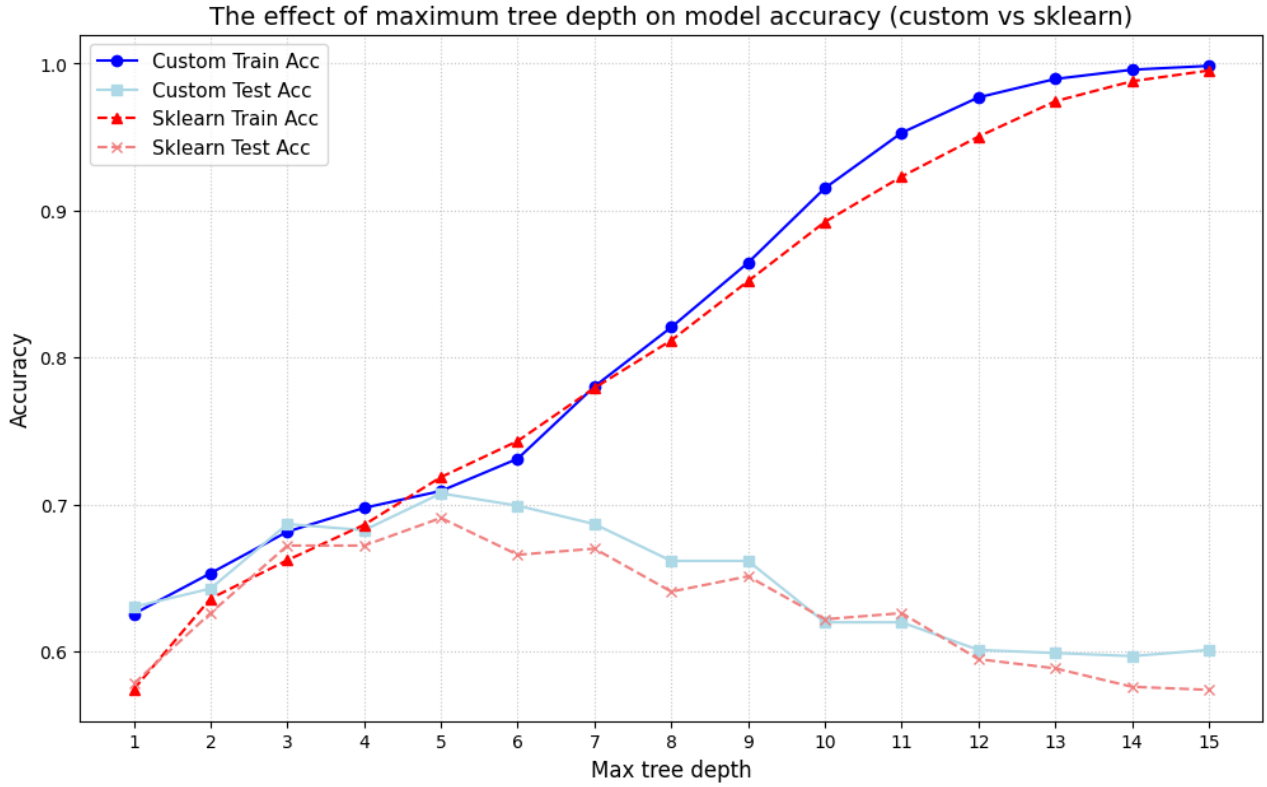
Confusion matrices (custom vs scikit-Learn)



Як видно з графіків, власні реалізації алгоритмів демонструють ідентичний або максимально наближений розподіл помилок порівняно з оптимізованими версіями зі scikit-learn.

Висновки щодо порівняльного аналізу класифікаторів:

1. **Decision Tree**: Алгоритм показав найкращу здатність до узагальнення серед усіх протестованих моделей. Обмеження гіперпараметрів (max_depth=5, min_samples_split=5) успішно запобігло перенавчанню на розріджених даних. Це підтверджується дослідженням впливу гіперпараметра max_depth на точність:



Графік демонструє, що після глибини дерева = 5 модель починає запам'ятовувати навчальну вибірку (точність прямує до 1.0), при цьому точність на тестовій вибірці падає. Поведінка власної моделі повністю корелює з поведінкою DecisionTreeClassifier зі sklearn.

2. **Gaussian Naive Bayes**: Модель має дещо нижчий показник точності, ніж попередня. Припущення про незалежність ознак допомогло впоратися з відносно високою розмірністю, проте загальна здатність моделі знаходити складні залежності є нижчою.

3. **k-Nearest Neighbors**: Незважаючи на коректне масштабування ознак (StandardScaler), kNN продемонстрував гірші результати через "прокляття розмірності". Наявність великої кількості закодованих категоріальних ознак розмила поняття Евклідової відстані, через що алгоритму складно знаходити дійсно релевантних "сусідів".

4. **1-Rule**: Абсолютний провал метрик 1-Rule став яскравим прикладом несумісності алгоритму та загального пайплайну препроцесингу. Через застосування StandardScaler до неперервних величин, алгоритм перенавчився на унікальних дробових значеннях тренувальної вибірки (кожен об'єкт став окремим правилом). На тестовій вибірці, зіткнувшись із невідомими значеннями, модель скотилася до передбачення виключно мажоритарного класу (4), що підтверджується нульовими значеннями Precision та Recall для інших класів. Це доводить, що для 1-Rule необхідна обов'язкова дискретизація (binning) числових ознак.

   У якості аналога 1-Rule зі sklearn було використано DecisionTreeClassifier з max_depth=1. Оскільки дерево рішень зі sklearn під капотом самостійно знаходить оптимальні пороги (thresholds) для неперервних величин, воно не страждає від проблеми унікальних дробових значень після StandardScaler, на відміну від базової реалізації 1-Rule.

Загальний висновок:

Самостійна програмна реалізація дозволила глибше зрозуміти математичний апарат алгоритмів та вузькі місця в їхній продуктивності. Для реальних задач на складних наборах даних краще підійшли б ансамблеві моделі на базі дерев рішень або оптимізовані реалізації метричних алгоритмів.

Посилання на дані:

https://www.kaggle.com/datasets/rabieelkharoua/students-performance-dataset